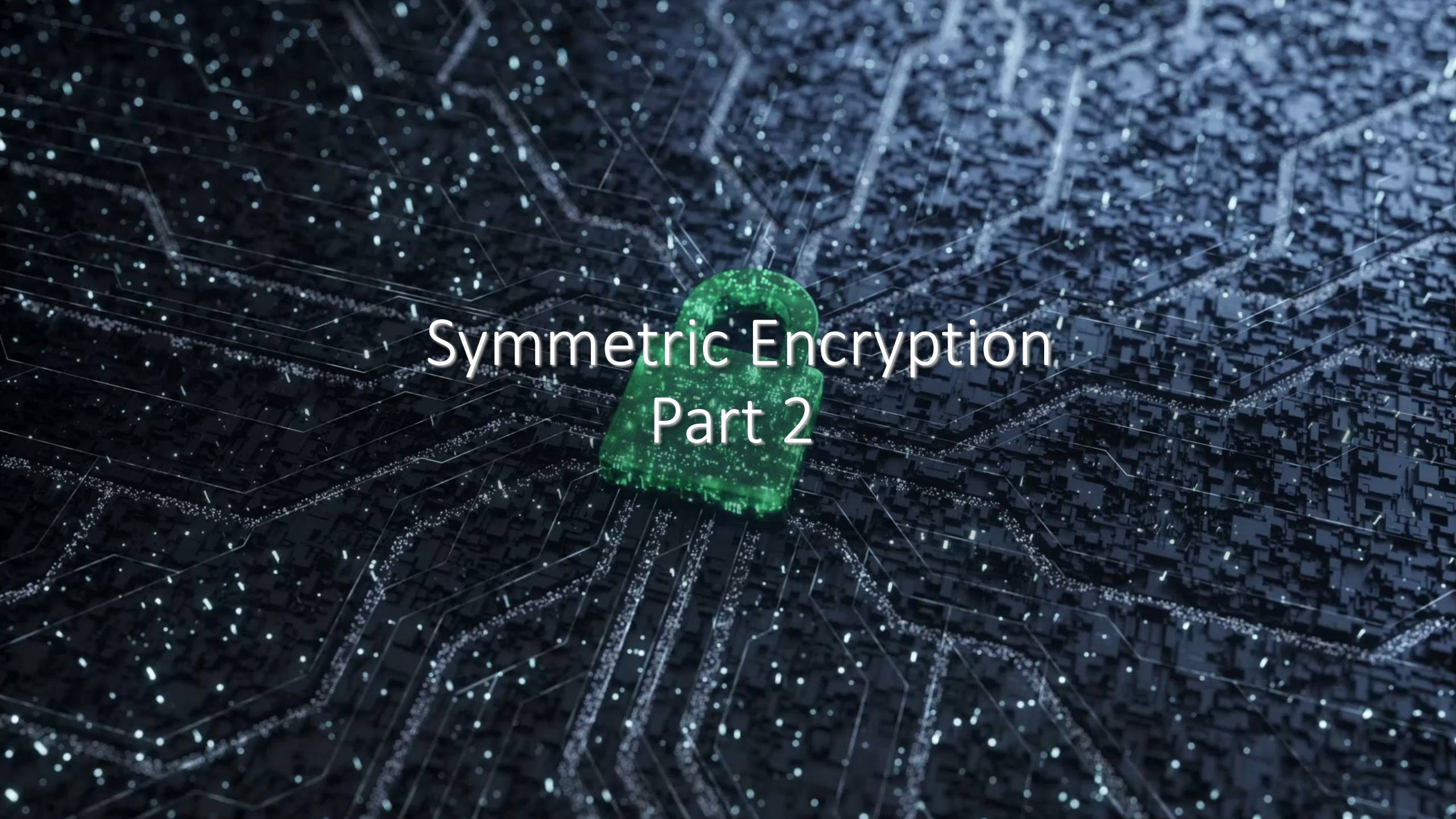
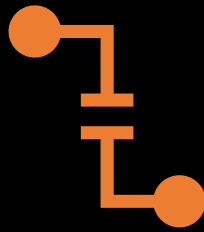


The background is a dark blue, almost black, surface covered with a dense, intricate network of glowing white and light blue lines. These lines form a complex, organic circuit-like pattern that resembles a microscopic view of a microchip or a neural network. The lines vary in thickness and brightness, creating a sense of depth and movement. In the center of the image, there is a glowing green padlock. The padlock is semi-transparent, allowing the circuit lines to be seen through it. It has a classic shape with a circular shackle at the top and a rectangular body. The green glow is bright and slightly grainy, giving it a digital or futuristic appearance. Overlaid on the padlock and the surrounding circuitry is the text "Symmetric Encryption" in a large, white, sans-serif font. Below it, "Part 2" is written in a slightly smaller, similar font. The text is centered horizontally and partially overlaps the padlock.

Symmetric Encryption

Part 2

Block vs Stream



Block cipher

Input is blocks of elements – transformed to the output block all at once



Stream cipher

Process the elements continuously, one element at a time

Block ciphers modes

- Block ciphers may be used in different modes. Most common modes are
- Electronic Codebook Mode (ECB)
- Cipher Block Chaining (CBC)
- Cipher Feedback Mode (CFB)

Electronic Codebook Mode (ECB)

- **Simple mode**: each block, say of size 64 bits is encrypted with the same key
- For a given block of the plaintext and a given key the result of encryption is **unique**
- If a block of plaintext is repeated several times, the result of encryption contains several copies of the same ciphertext
- So, the encryption of the lengthy (regular) messages **might be insecure**

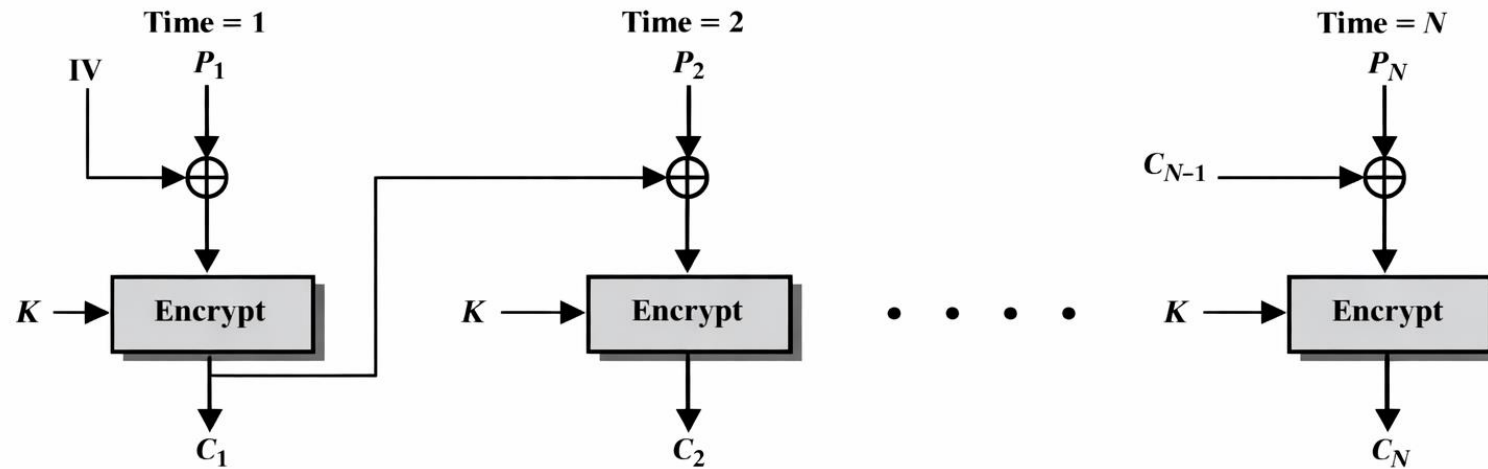
Cipher Block Chaining Mode (CBC)

- CBC mode fixes abovementioned disadvantage of ECB mode: here the **same** blocks of plaintext may produce **different** blocks of ciphertext
- **Simple idea:** before encryption, a block of the plaintext is XOR'ed with the result of encryption of the previous block

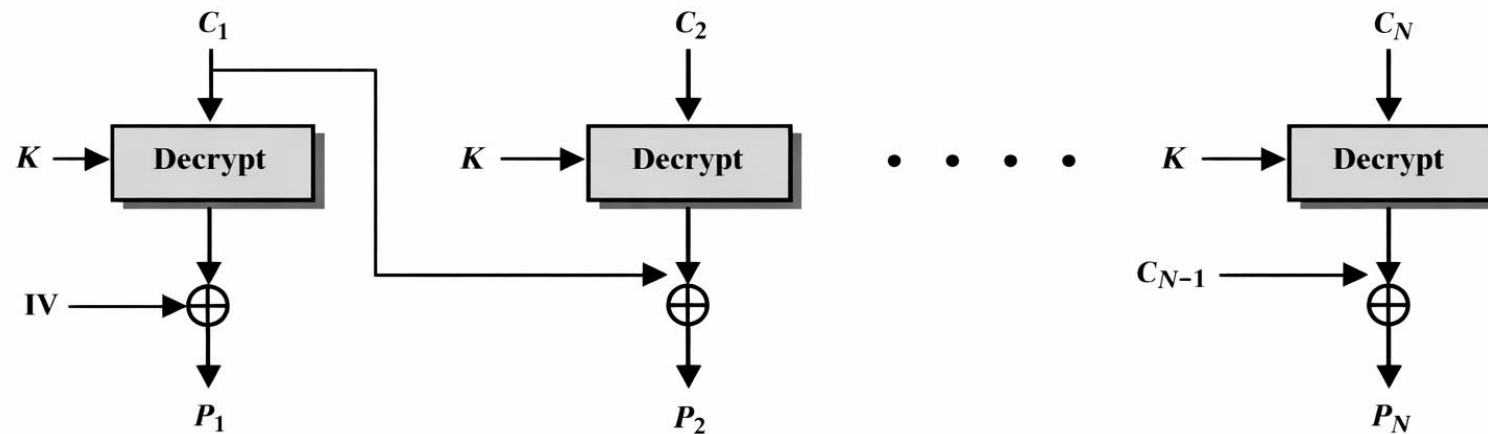
$$C_i = E_K[C_{i-1} \oplus P_i]$$

- For the first block encryption some initialisation vector (IV) is used
- It is better to keep both *the key and IV secret*

CBC encryption and decryption



(a) Encryption

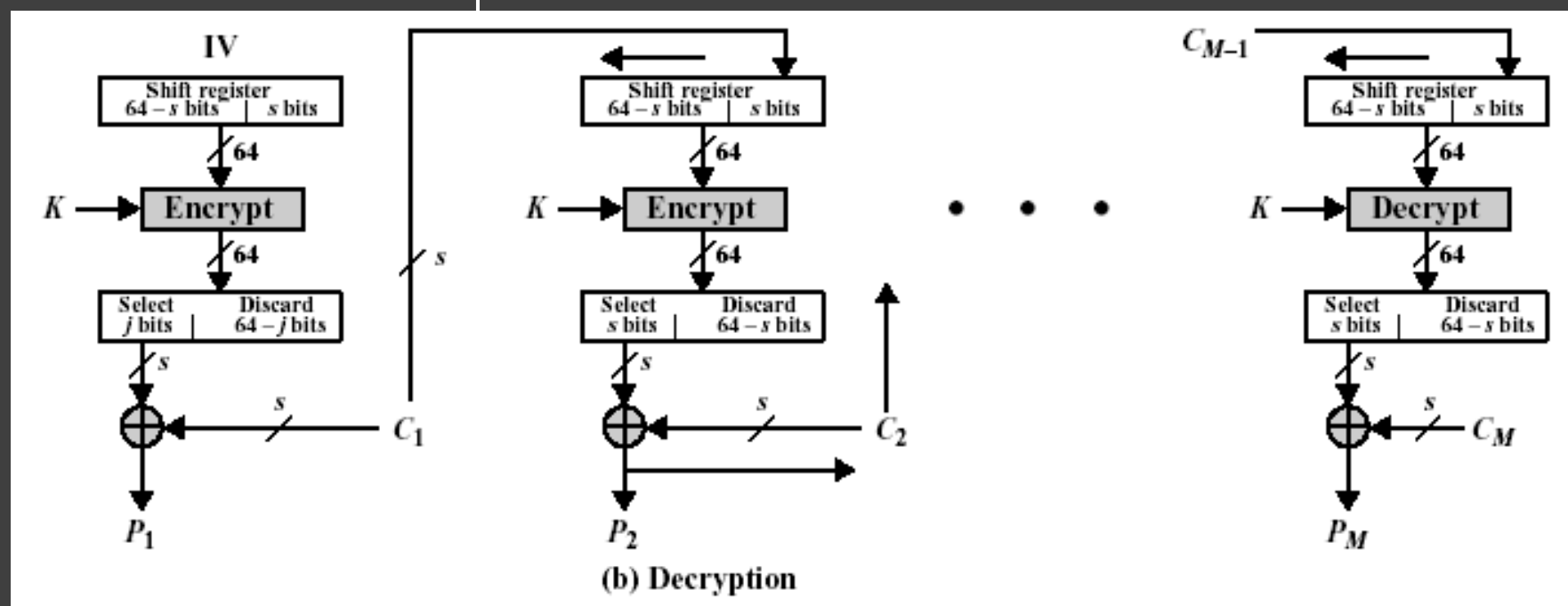
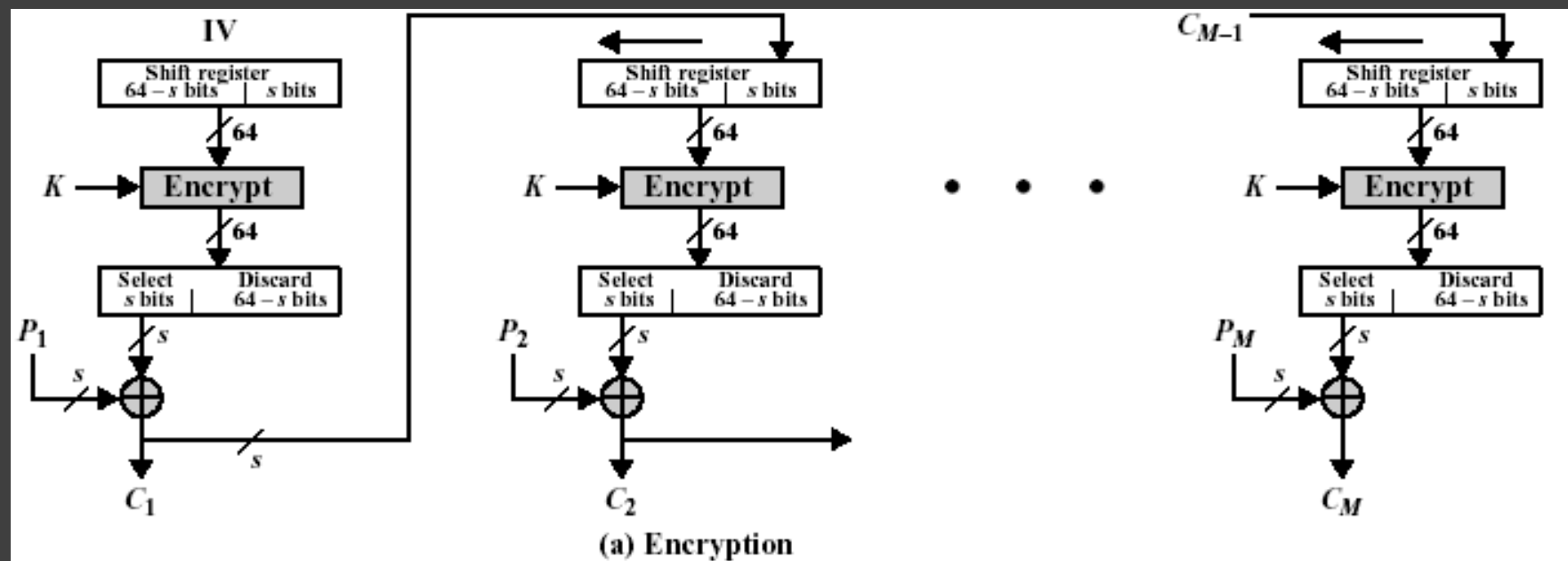


(b) Decryption

Cipher Feedback Mode (CFB)

- CFB mode may be used to transform a block cipher to the stream cipher
- It has a parameter s (the size of transmission unit); if 8-bit characters are used as transmission unit, then $s = 8$
- Shift register of size equal to the block-size of the block cipher is used (typically it is 64 bits)
- Again, an initialisation vector (IV) is needed

s-bits CFB encryption & decryption





Key distribution

- **From requirements for symmetric encryption:**
- “Sender and receiver must *have obtained copies of the secret key* in a secure way and must keep the key secure”
- **Important issue:** how to distribute secret keys?

Key distribution, manual delivery

- For two parties A and B:
- A key could be created by A and delivered physically to B (or vice versa);
- A key could be created by the third trusted party C and delivered physically to A and B;
- Difficult to use in wide area distributed systems, when dynamic connections are needed.

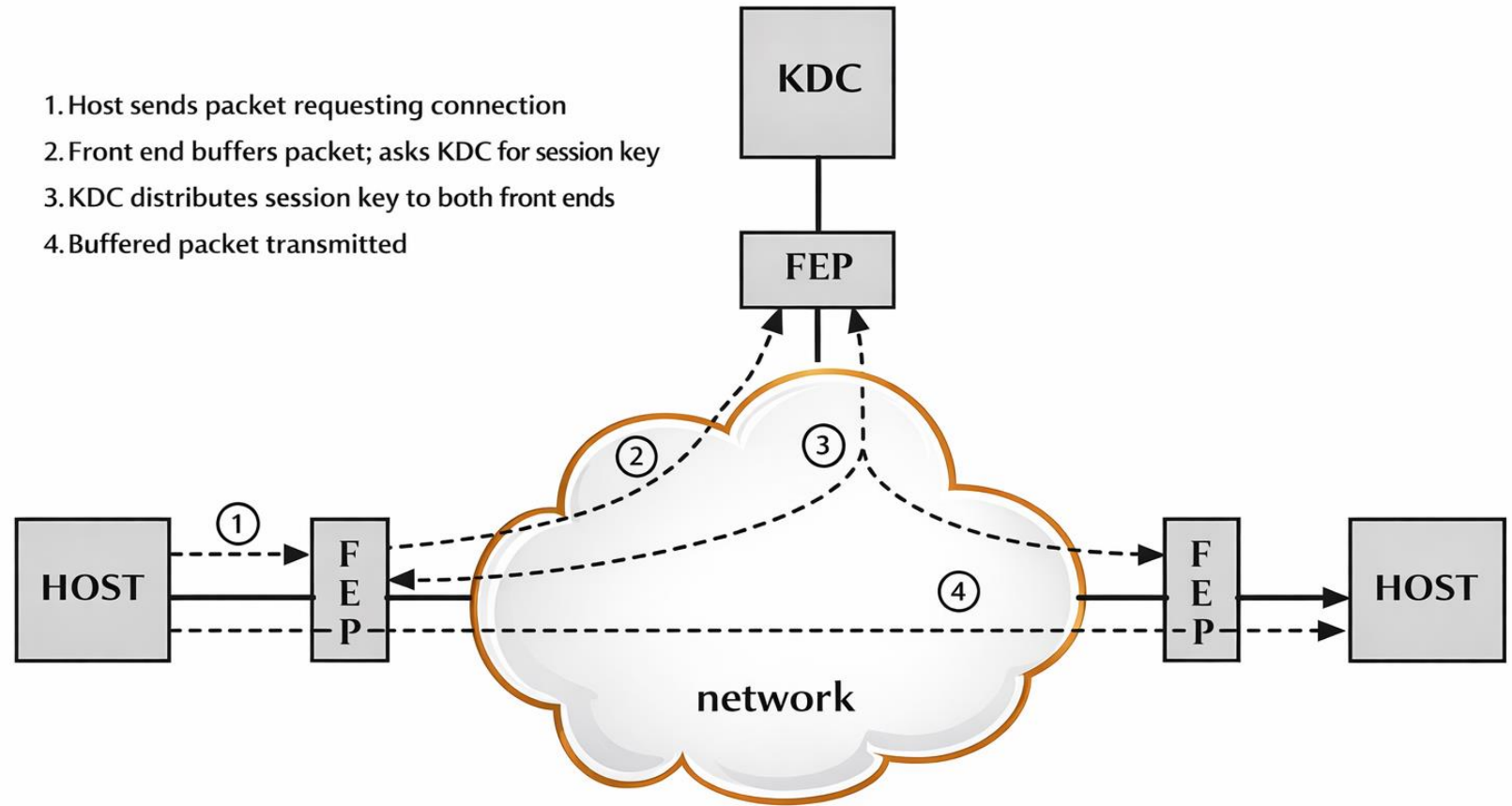


Key distribution, further techniques

- If A and B have used recently a secret key, one of them could create a new secret key and send it to the partner using old key;
- *Potential problem: once an attacker learned one key, he can disclose all keys afterwards*
- There is a third trusted party C connected by encrypted channels with both A and B. Then C creates a key and distributes it among A and B using encrypted channels

Automated key distribution

1. Host sends packet requesting connection
2. Front end buffers packet; asks KDC for session key
3. KDC distributes session key to both front ends
4. Buffered packet transmitted



- FEP = Front End Processor
- KDC = Key Distribution Center